COS30018 - Option C - Task 7 Report

Extension

Student Name: Kha Anh Nguyen
Student ID: 103814796
Date: October 28, 2025
Course: COS30018 - Intelligent Systems
Project: FinTech101 Stock Price Prediction System
Github repo: Leissha/stock_prediction

Table of Contents

---

1. Key changes:

Criterion	Section
Data Collection & Preprocessing	Section 2: *Google News RSS + Yahoo Finance, time alignment and caching*
Sentiment Analysis	Section 3: *FinBERT sentiment scoring with daily aggregation*
Feature Engineering & Modelling	Section 4: *OHLCV + sentiment features, Sklearn ML models*
Evaluation	Section 5: *Classification metrics (Accuracy, Precision, Recall, F1, Confusion Matrix) with baseline comparison*
Independent Research Component	Section 6, some more enhancements:

	• Model architecture: *Hybrid DL architecture (CNN-Time series, Attention-Time series) & batch evaluation.* • Feature engineering: Add social media, sentiment rolling, technical indicators features
Reporting	Section 7: This document + Github (*architecture.md, README)*

2. Data Collection & Preprocessing

2.1 News Data Collection

Article ref**:** *https://medium.com/@wl8380/how-to-get-historic-stock-news-for-free-with-python-a-step-by-step-guide-02276d3c4860*

Code ref: *https://colab.research.google.com/drive/1OzKKKco-hx7CoH_uP1M2L7FsuLkcfu9l#scrollTo=yq064Ti9D2oL*

The article guided us to use Google News RSS with public, stable, time-stamped and no API key and date limitation. We can simply query "Company OR Ticker" within a date window and parse items.

The news fetching code is located at sentiment/crawl_news.py:

```
def _fetch_news_range(ticker: str, start_date: datetime, end_date: datetime)
    # Detect Australian stocks and use AU locale for better coverage
    # CBA.AX, WBC.AX, etc. will get Australian news sources
    is_australian = ticker.endswith('.AX') or ticker.upper().endswith('.AX')

    if is_australian:
        # Australian locale for better local news coverage
        rss_url = f"https://news.google.com/rss/search?q={query}&hl=en-AU&gl=AU&ceid=AU:en"
    else:
        # Default to US locale for international stocks
        rss_url = f"https://news.google.com/rss/search?q={query}&hl=en-US&gl=US&ceid=US:en"

    feed = feedparser.parse(rss_url)
    rows = []
    for entry in feed.entries:
        parsed_date = datetime(*entry.published_parsed[:6])
        rows.append({
            'date': parsed_date.strftime('%Y-%m-%d %H:%M:%S'),
            'title': entry.title,
            'url': entry.link,
            'source': entry.get('source', {}).get('title', 'N/A'),
            'company': company_name,
            'ticker': ticker
```

```
        })
    return pd.DataFrame(rows)
```

sentiment/crawl_news.py

2.2 Caching System with unique identifier search (mocking DB)

I also cache the scraped news by appending to the ticker CSV and avoid duplication using url and title+date:

```
def _append_cache(file_path: Path, df: pd.DataFrame) -> None:
    if file_path.exists():
        existing = pd.read_csv(file_path)
        df = pd.concat([existing, df], axis=0, ignore_index=True)

    # Deduplicate by URL (imitates DB primary key)
    if 'url' in df.columns:
        df = df.drop_duplicates(subset=['url'])
    # Extra careful with (title & date)
    df = df.drop_duplicates(subset=['title','date'])
    df.to_csv(file_path, index=False)
```

sentiment/sentiment_cache.py

Cache Structure:

2.3 Time Alignment & Preprocessing

When **merging** into the **main** OHCLV data pipeline, we encounter challenge when **news** articles published at **any time** (24/7), while **stock** pricing only open between **9:30am-4pm trading days** only. Hence, one solution is to do **daily aggregation** with **forward-fill** to handle NaNs (no backward to ensure data integrity). Note that with ffill, we assume the news effect carries on forward until new information arrives.

```
    def integrate_sentiment(self, df: pd.DataFrame, ticker: str, start_date: str, end_date:
str, source: str = 'all', include_social: bool = False) -> pd.DataFrame:
        """
```

```
        Integrate sentiment features into stock data.
        Time Alignment Strategy:
            1. Group news by calendar day (date_only)
            2. Merge with stock data (left join to preserve all trading days)
            3. Forward-fill sentiment for days without news (no backward fill to prevent data
leakage)
            4. Fill remaining NaNs with 0 (neutral sentiment, no news)
        Args:
            df: Stock data DataFrame
            ticker: Stock ticker symbol
            start_date: Start date (YYYY-MM-DD)
            end_date: End date (YYYY-MM-DD)
            source: News source ('all', 'google', 'yahoo', 'businesstoday'). Default: 'all'

        Returns:
            DataFrame with sentiment features added
        """
        # Get daily sentiment (optionally including social media)
        news_df, daily_sentiment = self.build_daily_sentiment(ticker, start_date, end_date,
source=source, include_social=include_social)

        if news_df.empty or daily_sentiment.empty:
            warning(f"No sentiment data found for {ticker}")
            return df

        # Merge with stock data
        df_with_sentiment = df.copy()

        # Create date_only column for merging
        df_with_sentiment['date_only'] = df_with_sentiment.index.date

        # Merge sentiment data
        df_with_sentiment = df_with_sentiment.merge(
            daily_sentiment,
            left_on='date_only',
            right_index=True,
            how='left'
        )

        # Get all sentiment column names (exclude date_only)
        sentiment_cols = [col for col in daily_sentiment.columns if col != 'date_only']

        # Forward-fill missing sentiment for all features
        df_with_sentiment[sentiment_cols] =
df_with_sentiment[sentiment_cols].ffill().fillna(0)
```

```
        # Drop temporary column
        df_with_sentiment = df_with_sentiment.drop('date_only', axis=1)

        # Count articles with valid sentiment scores
        articles_with_sentiment = news_df['sentiment_score'].notna().sum() if
'sentiment_score' in news_df.columns else 0

        return df_with_sentiment
```

sentiment/crawl_news.py

3. Sentiment Analysis

3.1 FinBERT Integration

At *sentiment/sentiment_analyzer.py*, I use ProsusAI/finbert from Hugging Face, the model reputed for financial domain BERT.

1. **Sentiment Score Range:** [-1, +1]
    - **Formula:** score = P(positive) - P(negative)
    - **Interpretation:** -1 (very negative), 0 (neutral), +1 (very positive)

Firstly, we create FinBERT class:

```
class FinBERTAnalyzer:
    def __init__(...):
        self.model_name = "ProsusAI/finbert"
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.label_mapping = {
              0: 'positive',
              1: 'negative',
              2: 'neutral'
         }
        self._load_model()
```

And load tokenizer

```
    def _load_model(self):
        """Load the FinBERT model and tokenizer."""
        if not TORCH_AVAILABLE:
            raise ImportError("PyTorch and transformers not available. Install dependencies to
use FinBERT.")


        # Load tokenizer
```

```
        self.tokenizer = AutoTokenizer.from_pretrained(self.model_name)
        # Prefer safetensors (avoids torch.load CVE requirement)
        self.model = AutoModelForSequenceClassification.from_pretrained(
            self.model_name,
            use_safetensors=True,
        )
        self.model.to(self.device)
        self.model.eval() # Set to evaluation mode
        logger.info(f"FinBERT model loaded successfully on {self.device}")
```

Single text inference function:

```
    def analyze_text(self, text: str) -> Dict[str, float]:
        """
        Analyze sentiment of a single text.

        Returns:
            Dictionary with sentiment analysis results:
                - p_pos: Probability of positive sentiment
                - p_neg: Probability of negative sentiment
                - p_neu: Probability of neutral sentiment
                - sentiment_score: Float in [-1, +1]
                - predicted_class: 'positive', 'negative', or 'neutral'
                - confidence: Float in [0, 1]
        """
        if self.tokenizer is None or self.model is None:
            raise RuntimeError("FinBERT is not loaded")
        # Tokenize text (truncate to 512 for BERT)
        inputs = self.tokenizer(
            text,
            return_tensors="pt",
            truncation=True,
            padding=True,
            max_length=512
        )

        # Move to device
        inputs = {k: v.to(self.device) for k, v in inputs.items()}

        # Inference (no gradient computation)

        with torch.no_grad():
            outputs = self.model(**inputs)
            predictions = torch.nn.functional.softmax(outputs.logits, dim=-1)

        # Extract probabilities
```

```
        probs = predictions[0].detach().cpu().numpy()

        return {
            'p_pos': float(probs[0]),  # positive
            'p_neg': float(probs[1]),  # negative
            'p_neu': float(probs[2]),  # neutral
            'sentiment_score': float(probs[0] - probs[1]),  # p_pos - p_neg
            'predicted_class': int(np.argmax(probs)),
            'confidence': float(np.max(probs))
        }
```

Batch sentiment inference:

```
    def analyze_batch(self, texts: List[str]) -> List[Dict[str, float]]:
        """
        Analyze sentiment of multiple texts in batch.

        Args:
            texts: List of text strings to analyze
        Returns:
            List of dictionaries with sentiment analysis results for each text
        """

        results: List[Dict[str, float]] = []
        for text in texts:
            results.append(self.analyze_text(text))
        return results
```

3.2 Daily Sentiment Aggregation

At sentiment/sentiment_cache.py:

Sentiment score is aggregated per trading day into mean sentiment score and news count of the trading day.

- **sentiment_mean:** Average sentiment per day (signal strength)
- **news_count:** Number of articles per day (confidence indicator):
    - **High** news count (>5 articles) = high confidence
    - **Low** news count (1-2 articles) = low confidence, noise possible
    - **Zero** news count = carry forward last sentiment

```
def build_daily_sentiment(self, ticker: str, start_date: str, end_date: str) ->
Tuple[pd.DataFrame, pd.DataFrame]:
    """Build daily sentiment aggregation from individual news articles.

    Aggregation Strategy:
```

```
    1. Group articles by calendar day (date_only)
    2. Calculate mean sentiment (equal weighting across articles)
    3. Count number of articles (confidence indicator)

    Returns:
        Tuple of (news_df, daily_sentiment_df)
        - news_df: Individual articles with sentiment scores
        - daily_sentiment_df: Aggregated daily sentiment with columns:
            - sentiment_mean: Average sentiment score
            - news_count: Number of articles (confidence indicator)
    """
    news_df = self.get_news_with_sentiment(ticker, start_date, end_date)
    valid_news = news_df.dropna(subset=['sentiment_score']).copy()

    if len(valid_news) > 0:
        # Group by date and aggregate
        daily = valid_news.groupby('date_only')['sentiment_score'].agg(['mean',
'count']).fillna(0)
        daily.columns = ['sentiment_mean', 'news_count']
    else:
        # No news available
        daily = pd.DataFrame(columns=['sentiment_mean', 'news_count'])

    return news_df, daily
```

3.3 Integration into the ML Pipeline

Same as other functions, the pipeline adds **sentimental** features only **when enabled** via **args**. Then dataframe will have 7 features: [OHLCV + sentiment_mean + news_count].

```
# Original features (from Yahoo Finance)
features = ['close', 'high', 'low', 'open', 'volume']  # 5 features

# Add sentiment features (if enabled)
if use_sentiment:
    sentiment_cache = get_sentiment_cache()
    df = sentiment_cache.integrate_sentiment(df, ticker, start_date, end_date)

    # Check if sentiment columns exist
    sent_extra = [c for c in ['sentiment_mean', 'news_count'] if c in df.columns]
    if sent_extra:
        features = features + sent_extra  # 7 features total
        print(f"Features: {features}")
```

finBERT check:

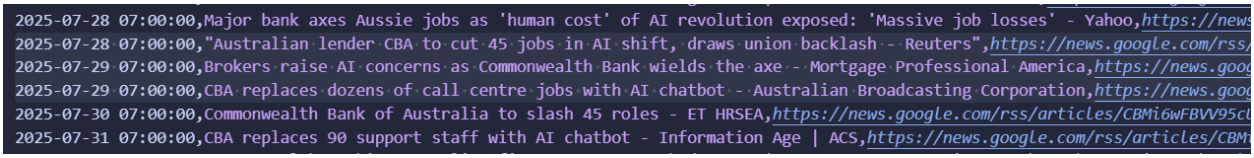

Figure 1. CBA News title

Figure 2. CBA news sentiment analysis

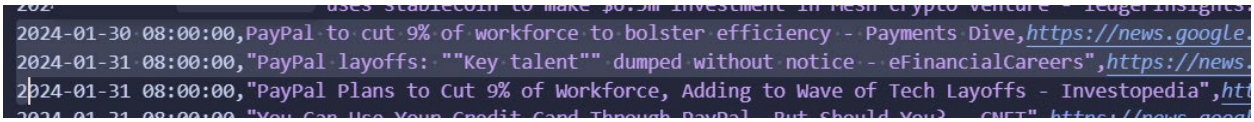

Figure 3. Paypal news title

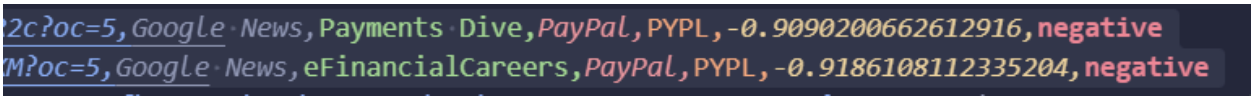

Figure 4. Paypal news sentiment analysis

FinBERT works as expected, even though the current approach ensures flexibility for multiple data and ticker source as we are getting information from google knowledge base, the search might be biased and limited with the hard-coded quote (Google News RSS query "{Company} OR {Ticker}").

However, as of testing with various sources like Yahoo news, NewsAPI, etc, this method helps encounter the long time range problem and cover most of the titles from Yahoo news and NewsAPI while free to use.

4. Classification evaluation

We'll add **sklearn ML models** for **binary** stock up/down signal **classification.** I also add BCE loss & binary prediction to **adapt current TFModel** (LSTM/ GRU/ RNN/ BiLSTM) for binary classification.

Main.py integration:

- For separate of concern, classification has a separate pipeline and will be invoked using "--classification" parse args flag.

```
def main():
```

```
    """Main execution function"""
    # Parse arguments and setup configuration
    args = parse_args()
    config = update_config_from_args(args)

    # Validate configuration
    errors = validate_current_config()
    # Ensure directories exist and print configuration
    ensure_directories()
    print_current_config()

    # Classification short-circuit (prepare once in main and reuse)
    if config.mode == "classification":
        run_classification_evaluation(config)
        return

    # Otherwise we run regression pipeline
    # Preprocess data
    bundle = prepare_data( #args  )
    # Generate meta path and inspection plots
    meta_path = generate_meta_path(config)
    generate_inspection_plots(config, bundle, meta_path)

    # Create and train/load model
    model = create_model(config, bundle, meta_path, args)
    # Generate predictions and evaluate
    run_regression_evaluation(model, bundle, args, meta_path)
```

main.py

The “--classification” flag would make a short cut and call the classification pipeline directly:

```
class ClassificationEvaluator:
    """
    Binary classification with multiple models and yes/no sentiment comparison.
    """

    def __init__(self, config):
        """Initialize with parsed PipelineConfig from main."""
        # Load main args config

    def evaluate(self):
        """Single method: data -> training -> evaluation -> non-sentiment comparison ->
results"""
        # 1. Prepare data
        bundle = self._prepare_data()
```

```
        self.bundle = bundle  # Store for plotting

        # 3. Train models
        results, y_test_binary, primary_pred = self._train_models(bundle)
        self.y_test_binary = y_test_binary  # Store for plotting
        self.primary_pred = primary_pred  # Store for plotting

        # 4. Run non-sentiment comparison
        baseline_results = self._run_non_sentiment_comparison()

        # 5. Save results
        self._save_results(results, baseline_results)
```

models/TFModel.py

ClassificationEvaluator pipeline includes:

4.1 ML for classification

We'll make some ML theory in lecture 5 into life, we'll use Sklearn models: *Logistic Regression, Random Forest, SVM* with balanced sampling option for less bias classification.

- **Logistic Regression:**
  - Uses **sigmoid** activation, optimized for binary classification
  - Use log loss (logistic loss) for classification loss
- **Random Forest:**
  - Decision tree ensemble with class balancing
  - Gini impurity or entropy for classification loss
- **SVM**:
  - Support Vector Machine with probability outputs
  - Hinge loss for classification loss

Our stock data has slight class imbalance (e.g., 52% Up, 48% Down) => We can use class_weight='balanced' to penalize misclassifications of minority class more heavily

```
def _train_models(self, bundle):
  sklearn_models = {
    'Logistic Regression': LogisticRegression(
        random_state=42, max_iter=1000, class_weight='balanced'
    ),
    'Random Forest': RandomForestClassifier(
        random_state=42, n_estimators=100, class_weight='balanced'
    ),
    'SVM': SVC(
```

```
        random_state=42, probability=True, class_weight='balanced'
    )
  }
  for model_name, model in sklearn_models.items():
    print(f"  Training {model_name}...")
    pred = self._train_and_predict_sklearn(model, bundle, y_train_binary)
    results[model_name] = self._evaluate_classification(y_test_binary, pred, model_name)
```

eval/classification_evaluator.py

Sklearn train/predict adapter since current preprocessed data was 3D:

```
def _train_and_predict_sklearn(self, model, bundle, y_train_binary):
    """Train and predict using sklearn model."""
    # Reshape data for sklearn
    X_train_flat = bundle.X_train.reshape(bundle.X_train.shape[0], -1)
    X_test_flat = bundle.X_test.reshape(bundle.X_test.shape[0], -1)
    model.fit(X_train_flat, y_train_binary.ravel())
    pred = model.predict(X_test_flat)
    return pred
```

eval/classification_evaluator.py

4.2 TFModel reuse

I reuse the args parse to allow us to choose any of the 4 TF models for the baseline.
Eg: *python main.py --classification --company CBA.AX --**model_name rnn** --use_sentiment*
Otherwise, it will auto select LSTM by default.

```
def _train_models(self, bundle):
  pred = self._train_tf_model(
    model_name=self.baseline_model.lower(),
    bundle=bundle,
    config=self.config,
    meta_suffix="classification",
  )
  results[self.baseline_model] = self._evaluate_classification(y_test_binary, pred,
self.baseline_model)
```

models/TFModel.py

To classify up/down for current TFModel we need to **convert** the **regression prediction** to **binary**:

```
def _to_binary(self, y, scaler=None):
    """Convert to binary labels (0/1)."""
    if scaler is not None:
        # Descale if scaler is present
```

```
        y_descaled = scaler.inverse_transform(y)
        return (y_descaled > 0).astype(int)
    else:
        return (y > 0).astype(int)
```

models/TFModel.py

Also add condition to use **Binary Cross-Entropy** for **loss**:

```
def fit(self, x_train, y_train, epochs=25, batch_size=32, learning_rate=1e-3, optimizer: str =
'huber', validation_data=None, meta_path=None, class_weight=None):
        """
        Train TensorFlow model
        validation_data: optional to monitor validation loss.
        """
        # Compile model
        optimizers = {'adam': Adam, 'rmsprop': RMSprop, 'sgd': SGD}
        opt_cls = optimizers[optimizer.lower()]
        opt = opt_cls(learning_rate=learning_rate)

        if self.task_type == 'binary':
            self.model.compile(optimizer=opt, loss='binary_crossentropy',
metrics=['accuracy'])
        else:
            # Prefer MAE to prevent collapsing to near-zero mean under MSE
            self.model.compile(optimizer=opt, loss='mean_absolute_error', metrics=['mae'])
```

models/TFModel.py

5. Evaluation

Section 5 implements traditional binary classification where models directly learn to classify stock movements as "Up" (1) or "Down" (0).

5.1 Binary classification comparison

Classification models are evaluated using accuracy, precision, recall, F1, confusion matrix.

```
def _evaluate_classification(self, y_true, y_pred, model_name="Model"):
    """Evaluate binary classification performance.

    Metrics:
    1. Accuracy: Overall correctness
    2. Precision: % of predicted Ups that are actually Up
    3. Recall: % of actual Ups that model predicted as Up
    4. F1-Score: Harmonic mean of Precision and Recall
    5. Confusion Matrix: Breakdown of TP, TN, FP, FN
    """
```

```python
    # Calculate metrics
    accuracy = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, zero_division=0)
    recall = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)

    # Confusion matrix
    cm = confusion_matrix(y_true, y_pred)

    # Print confusion matrix
    print_confusion_matrix(cm, y_true)

    return {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1_score': f1,
        'confusion_matrix': cm
    }
```

eval/classification_evaluator.py

COOKBOOK:

Confusion Matrix Trading Interpretation:

		Predicted	
		DOWN	*UP*
Actual	*DOWN*	*TN*	*FP (Wrong "Buy" signal (lose money))*
	UP	*FN (Missed)*	*TP (True Positive: Correct "Buy" signal (make money))*

Metric Interpretation for Trading:

Metric	**Formula**	**Trading Interpretation (What the metrics conveys)**
Accuracy	*(TP + TN) / Total*	Overall prediction correctness (not ideal for imbalanced data 😟)
Precision	*TP / (TP + FP)*	Of all "Buy" signals, what % are actually profitable?
Recall	*TP / (TP + FN)*	Of all profitable days, what % did we catch?
F1-Score	*2 × (Precision × Recall) / (Precision + Recall)*	Balance between precision and recall (penalizes extreme values)

5.2 Baseline vs Sentiment Comparison

Reuse the config to train non-sentiment TFModel and compare sentiment effectiveness.

```python
def _run_non_sentiment_comparison(self):
    """Train baseline model WITHOUT sentiment features for comparison.

    Strategy:
    1. Clone current config
    2. Disable sentiment features (use_sentiment=False)
    3. Re-prepare data (OHLCV + Technical Indicators only, no sentiment)
    4. Train same model architecture (e.g., Attention-LSTM)
    5. Compare performance metrics
    """
    import copy
    baseline_config = copy.deepcopy(self.config)
    baseline_config.data.use_sentiment = False

    # Prepare baseline data (no sentiment)
    bundle_baseline = prepare_data(
        ticker=baseline_config.data.company,
        start_date=baseline_config.data.start_date,
        end_date=baseline_config.data.end_date,
        lookback=baseline_config.data.lookback,
        horizon=baseline_config.data.horizon,
        split_ratio=baseline_config.data.split_ratio,
        use_sentiment=False,  # KEY: Disable sentiment
        target_as_return=baseline_config.data.target_as_return,
    )

    # Train baseline model using same architecture
    baseline_pred = self._train_tf_model(
        model_name=self.baseline_model.lower(),
        bundle=bundle_baseline,
        config=baseline_config,
        meta_suffix="baseline_classification",
    )

    y_test_baseline = self._to_binary(bundle_baseline.y_test, bundle_baseline.target_scaler)
    return self._evaluate_classification(y_test_baseline, baseline_pred, f"Baseline
{self.baseline_model} (No Sentiment)")
```

eval/classification_evaluator.py

Example classification run:

- Once called, the pipeline will then run all ML models training and evaluate against sentiment & non-sentiment baseline DL model.

Below is an example classification pipeline execution:

python main.py --classification --company CBA.AX --model_name rnn --use_sentiment --target_ret

```
INFO:sentiment.sentiment_analyzer:FinBERT model loaded successfully on cuda
============================================================
PIPELINE CONFIGURATION
============================================================
Execution Mode: CLASSIFICATION
Company: PYPL
Date Range: 2023-11-01 to 2025-10-31
Target Feature: Close
Lookback: 60
Horizon: 1
Test Size: 0.2
Validation Size: 0.2
Scaling: Enabled
Sentiment: Disabled
Classification Models: LSTM, Logistic Regression, Random Forest, SVM
============================================================
```

Figure 1. Training configuration

```
Preparing data for PYPL classification...
Loading cached data for PYPL
  Integrating sentiment features (source: google)...
Fetching new articles for PYPL
  Using cached news for PYPL; range already covered
Sentiment data cached for PYPL
Added 2 sentiment features
News articles processed: 200
Days with news: 103

Full DataFrame (tail 5):
               close       high        low       open    volume  close_return  sentiment_mean  news_count
Date
2025-10-03  69.250000  69.529999  67.750000  68.099998  12219800      0.004642       -0.217011         2.0
2025-10-06  71.290001  71.870003  69.452003  70.220001  16769900      0.029458        0.157048         4.0
2025-10-07  74.610001  75.675003  73.059998  74.070000  31977500      0.046570        0.389495         6.0
2025-10-08  76.129997  76.500000  73.275002  75.235001  17904800      0.020373        0.055462         1.0
2025-10-09  75.750000  77.339996  75.032997  77.190002  15774400     -0.004991       -0.038402         2.0

[Preview] Train scaled features (head 3):
      close      high       low      open    volume  sentiment_mean  news_count
0 -1.262844 -1.303956 -1.438688 -1.262431  2.967422       -0.232302    5.880490
1 -1.157961 -1.219794 -1.150586 -1.224963  0.728147       -0.138713    1.517352
2 -1.307218 -1.174707 -1.272122 -1.100406  0.526508        0.210263   -0.227903
DataBundle[PYPL]
  Train: X(230, 60, 7) → y(230, 1)
  Val  : X(96, 60, 7) → y(96, 1)
  Test : X(96, 60, 7) → y(96, 1)
  Mode : return (log=False)
  Feats: 7 -> close, high, low, open, volume, sentiment_mean, news_count
  Target Debug:
    y_train: shape(5,), range[-5.348483, 3.922511]
    y_test:  shape(5,), range[-4.138494, 2.085896]
    sample_train: [-1.8024237   0.4190269   0.20995228 -0.750794    1.557129  ]
    sample_test:  [-0.2221279  -1.1386583   0.99951047 -0.11777483 -0.42540434]
    scaler: True
```

Figure 2. Sentiment integration & news cache works (we can see the X.shape[2] = 7).

No-sentiment model training (binary classification)

```
============================================================
COMPARISON: SENTIMENT VS NON-SENTIMENT
============================================================
Loading cached data for PYPL

Full DataFrame (tail 5):
Price           close       high        low       open    volume  close_return
Date
2025-10-03  69.250000  69.529999  67.750000  68.099998  12219800      0.004642
2025-10-06  71.290001  71.870003  69.452003  70.220001  16769900      0.029458
2025-10-07  74.610001  75.675003  73.059998  74.070000  31977500      0.046570
2025-10-08  76.129997  76.500000  73.275002  75.235001  17904800      0.020373
2025-10-09  75.750000  77.339996  75.032997  77.190002  15774400     -0.004991

[Preview] Train scaled features (head 3):
      close      high       low      open    volume
0 -1.262844 -1.303956 -1.438688 -1.262431  2.967422
1 -1.157961 -1.219794 -1.150586 -1.224963  0.728147
2 -1.307218 -1.174707 -1.272122 -1.100406  0.526508
DataBundle[PYPL]
  Train: X(230, 60, 5) → y(230, 1)
  Val  : X(96, 60, 5) → y(96, 1)
  Test : X(96, 60, 5) → y(96, 1)
  Mode : return (log=False)
  Feats: 5 -> close, high, low, open, volume
  Target Debug:
    y_train: shape(5,), range[-5.348483, 3.922511]
    y_test:  shape(5,), range[-4.138494, 2.085896]
    sample_train: [-1.8024237   0.4190269   0.20995228 -0.750794    1.557129  ]
    sample_test:  [-0.2221279  -1.1386583   0.99951047 -0.11777483 -0.42540434]
    scaler: True
```

Figure 3. For non-sentiment model training, the pipeline reuses the cached data

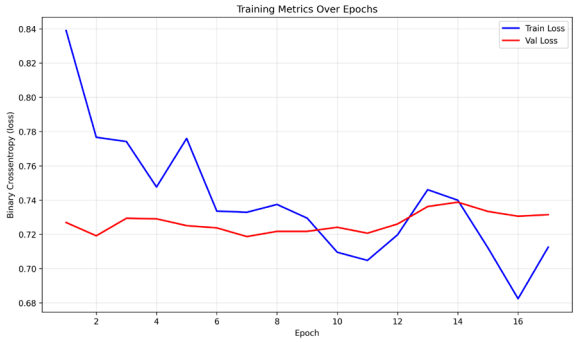

Figure 4. RNN training plot with sentiment

Figure 3. Non-sentiment training is smoother

Result:

```
------------------------------------------------------------
RNN                  0.5208     0.5275     0.9412     0.6761
Logistic Regression  0.6042     0.6857     0.4706     0.5581
Logistic Regression  0.6042     0.6857     0.4706     0.5581
Random Forest        0.5729     0.6042     0.5686     0.5859
SVM                  0.5312     0.7143     0.1961     0.3077

============================================================
SENTIMENT IMPACT ANALYSIS
============================================================
RNN                     Accuracy   Precision  Recall     F1-Score
------------------------------------------------------------
RNN + Sentiment          0.5208     0.5275     0.9412     0.6761
RNN Baseline             0.5208     0.5287     0.9020     0.6667

Sentiment Impact:
  Accuracy improvement: +0.0000 (+0.0%)
  F1-Score improvement:  +0.0094 (+0.9%)
```

Figure 4.

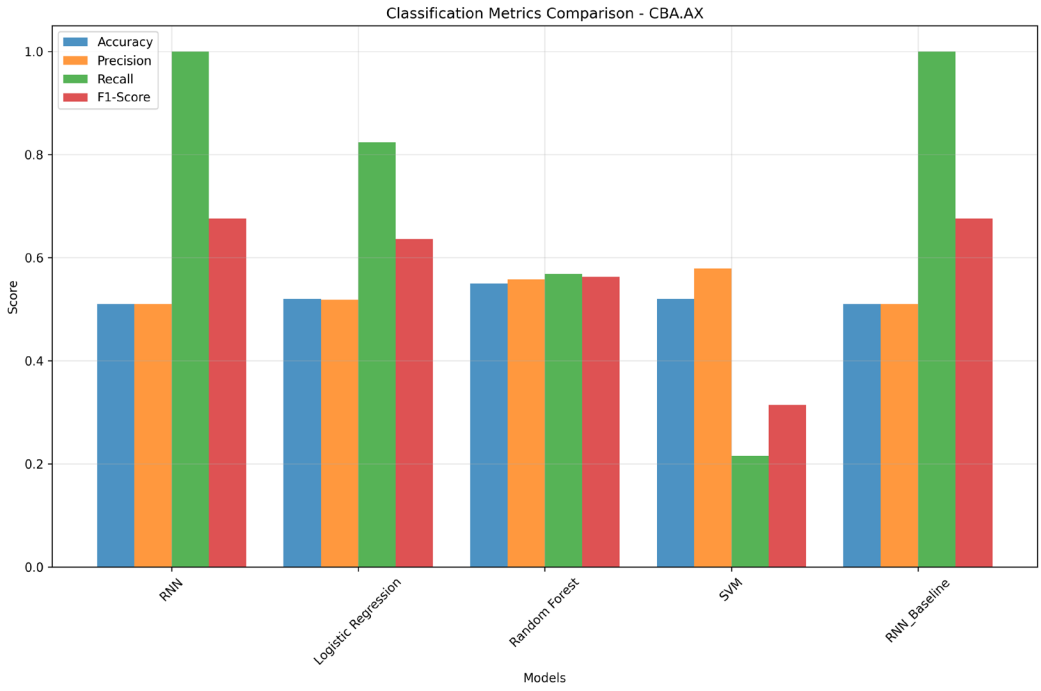

Figure 5. Metric scores bar chart

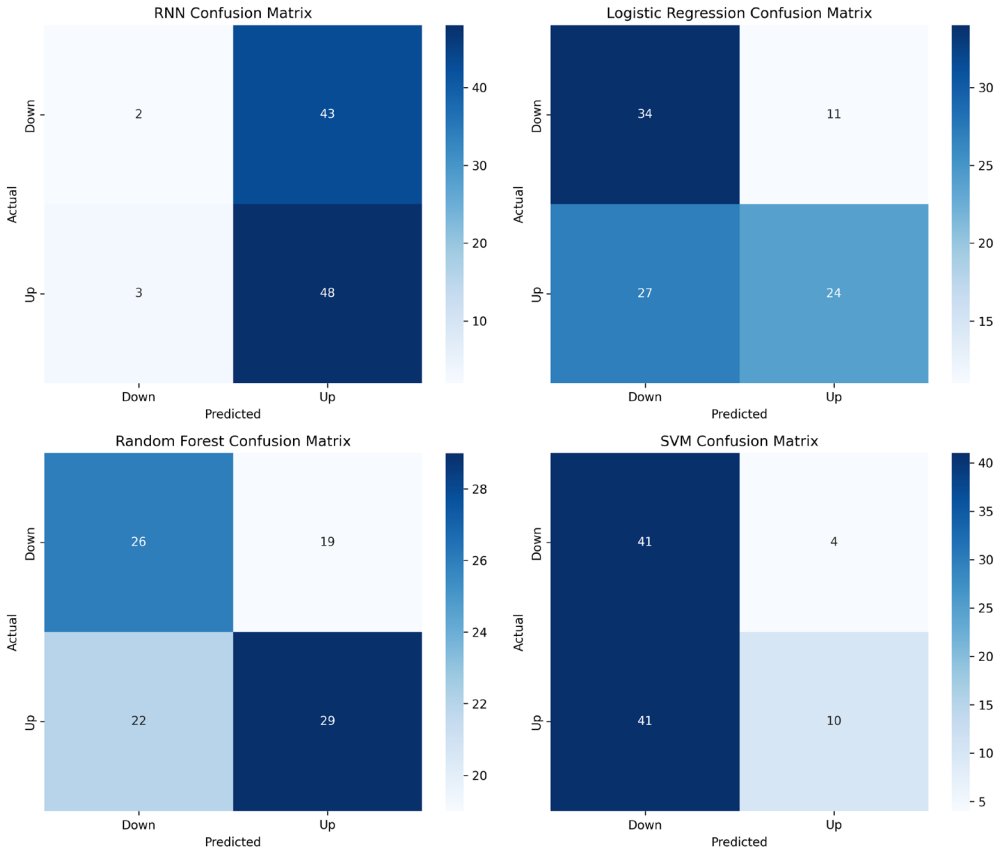

Figure 6. Confusion matrix

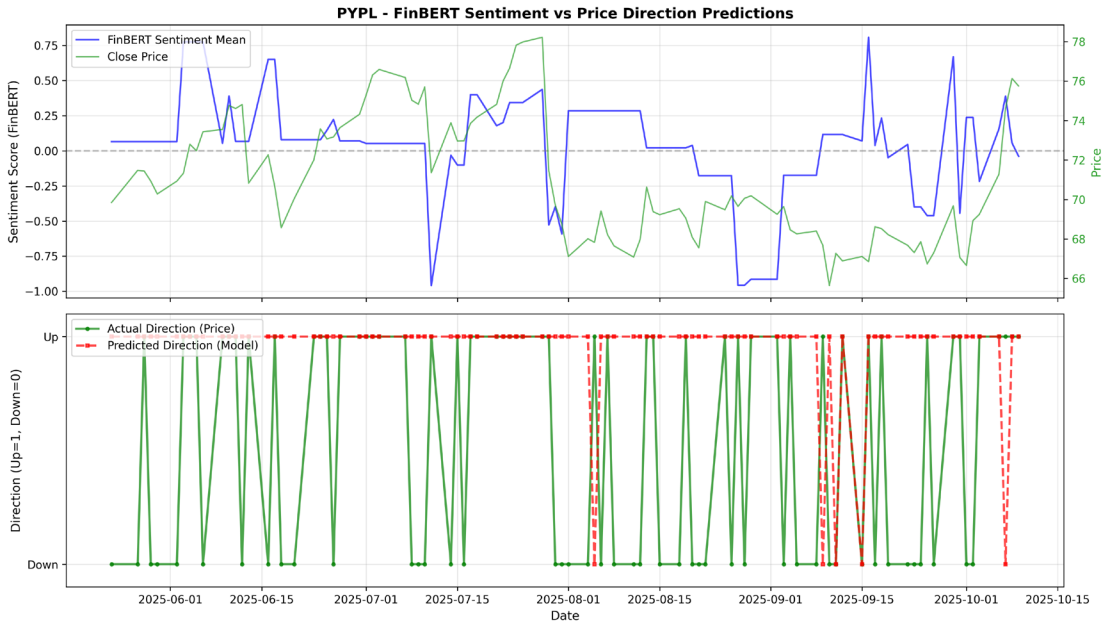

Figure 7. Paypal trial: Sentiment vs true price (up) & RNN pred vs true price (down)

Based on the confusion matrix, it is suggested that **DL model has collapsed** and is unable to **distinguish the up/down.** Adding **only 2 sentimental features** from specific news does **not help** much **but noise** as the stock prices encapsulate more complex factors.

Also, one hypothesis is that current **binary classification** approach **does not suit the time series stock problem**. **DL models are not optimized** to learn the **magnitude** of the prediction, **but just direction** and hence did not perform well in noticing the up/down trend.

Even though I have tried several sentiment sources (NewsAPI, yahoo news, etc), **most of the news derived from Google search** or restricted by the date range, so the current sentiment features aren't making any impactful improvements. A good news is the basic sentiment and ML models integration going on well based on the log.

Moving forward, I'll add **more data sources from social media** and do features engineering like **technical indicators** to help models learn the complex relationship between the sentiment & stock prices better. I will also **try new model architecture** to test the prediction performance in the next section.

1. **Model Collapse Issue:**
    - Some configurations (e.g., RNN with sentiment) show model collapse
    - **Evidence**: Confusion matrix shows model predicting mostly one class *(UP dominant on Figure 6-7)*
    - **Hypothesis**: Binary classification with BCE loss may not be optimal for this regression-natured problem
2. **Sentiment Signal Weakness:**

- Sentiment features alone (2 features) may be insufficient
- **Evidence**: Uncorrelate sentimental analysis vs stock price *(Figure 7)*
- **Conclusion**: Need stronger features (technical indicators) + better architectures

3. **Architecture Dependency:**
   - Different models respond differently to sentiment
   - **Hypothesis**: Attention mechanisms may better utilize sparse sentiment signals
   - **Next Step:** Section 6 explores **hybrid architectures** (CNN-RNN, Attention-RNN hybrid model) + **Social media + rolling sentiment** and **technical indicators** features + test if **regression return** model predicts up/down trend better (by converting to binary at prediction time).

6. Independent Research Component

Following the observations in Section 5 that sentiment features alone were insufficient and DL models showed collapsed predictions (unable to distinguish up/down movements). I tried another approach to see if **hybrid architectures** could improve classification performance beyond the baseline LSTM model.

Candidates model are CNN-LSTM hybrid and Attention-LSTM.

6.1 Feature Engineering Enhancement

Before testing the models, I enhance the current OHLCV features to include **technical indicators,** which are commonly used in quantitative finance:

```
def calculate_technical_indicators(df):
    """Calculate technical indicators commonly used in quantitative finance.

    Indicators:
    1. SMA (Simple Moving Average): Trend indicator
    2. EMA (Exponential Moving Average): Trend indicator with recent bias
    3. RSI (Relative Strength Index): Momentum oscillator (overbought/oversold)
    4. MACD (Moving Average Convergence Divergence): Trend + momentum

    Returns:
        DataFrame with 6 additional columns:
        [sma_20, ema_20, rsi_14, macd, macd_signal, macd_hist]
    """
    if 'close' in df.columns:
        close = df['close']

        # 1. Simple Moving Average (20-day)
        df['sma_20'] = close.rolling(window=20, min_periods=20).mean()

        # 2. Exponential Moving Average (20-day)
        df['ema_20'] = close.ewm(span=20, adjust=False).mean()
```

```
    # 3. Relative Strength Index (14-day)
    delta = close.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)
    avg_gain = gain.rolling(window=14, min_periods=14).mean()
    avg_loss = loss.rolling(window=14, min_periods=14).mean()
    rs = avg_gain / avg_loss.replace(0, np.nan)
    df['rsi_14'] = 100 - (100 / (1 + rs))

    # 4. MACD (12, 26, 9)
    ema12 = close.ewm(span=12, adjust=False).mean()
    ema26 = close.ewm(span=26, adjust=False).mean()
    macd = ema12 - ema26
    signal = macd.ewm(span=9, adjust=False).mean()
    df['macd'] = macd
    df['macd_signal'] = signal
    df['macd_hist'] = macd - signal

return df
```

The technical indicators are added after the data split, dataframe now has 11 features *(OHLCV + SMA_20, EMA_20, RSI_14, MACD, MACD_signal, MACD_hist)*. This provides richer representation of price patterns, momentum, and trend information for the models to learn from.

Technical Indicator Justification:

Indicator	Type	Formula	Trading Signal	Why Useful for ML
SMA (20-day)	Trend	Mean of 20-day close prices	Price > SMA → Bullish trend	Captures medium-term trend direction
EMA (20-day)	Trend	Weighted mean (recent prices weighted higher)	Price > EMA → Bullish with recent strength	More responsive than SMA to recent changes
RSI (14-day)	Momentum	100 - (100 / (1 + RS)), RS = Avg Gain / Avg Loss	RSI > 70 → Overbought (sell signal); RSI < 30 → Oversold (buy signal)	Identifies potential reversal points (overbought/oversold conditions)
MACD	Trend + Momentum	EMA(12) - EMA(26)	MACD > Signal → Bullish crossover	Combines trend strength and momentum changes
MACD Histogram	Momentum	MACD - MACD Signal	Positive histogram → Bullish momentum increasing	Visual representation of MACD convergence/divergence

Final Feature Set:

OHLCV (5) + Sentiment (2) + Technical Indicators (6) = 13 features total

Input Shape: (batch_size, 60 timesteps, 13 features)

```
[Preview] Train scaled features (head 3):
      close      high       low      open    volume  sma_20    ema_20  rsi_14      macd  macd_signal  macd_hist
0 -1.653059 -1.630425 -1.659669 -1.679179  0.275877     0.0 -1.586118     0.0 -1.176147    -1.310201  -0.007514
1 -1.675386 -1.663887 -1.674036 -1.636171 -0.213357     0.0 -1.588295     0.0 -1.203438    -1.316297  -0.061458
2 -1.638355 -1.641219 -1.634251 -1.638893 -0.597663     0.0 -1.586654     0.0 -1.179245    -1.315770  -0.002849
```

6.2: Social Media Integration (Reddit):

Reference:

https://medium.datadriveninvestor.com/predicting-market-sentiment-with-social-media-a-deep-learning-approach-to-fintech-trading-91993eca0af4

6.2.1 Reddit Sentiment Crawler

Reddit offers:

- **Free API:** PRAW (Python Reddit API Wrapper), no cost, 60 requests/minute
- **Retail Investor Sentiment:** Captures sentiment from r/wallstreetbets, r/investing, r/stocks
- **Rich Discussion Data:** Post titles + body text (up to 500 chars) for comprehensive sentiment analysis

Target Subreddits:

```python
# Reddit subreddits for stock discussions
STOCK_SUBREDDITS = [
    "wallstreetbets",      # Retail trading, high volume, meme stocks
    "investing",           # Long-term investors, fundamental analysis
    "stocks",              # General stock discussions
    "StockMarket",         # Market news and analysis
    "SecurityAnalysis",    # Fundamental analysis, value investing
    "ASX",                 # Australian Stock Exchange discussions
    "AusFinance",          # Australian finance and investing
]
```

sentiment/crawl_social.py

Implementation:

```python
def fetch_reddit_posts(ticker: str, start_date: datetime, end_date: datetime,
                       subreddits: Optional[List[str]] = None,
                       limit: int = 250) -> pd.DataFrame:
    """Fetch Reddit posts mentioning a ticker from specified subreddits.
```

```
Target Subreddits:
- r/wallstreetbets: Retail trading, meme stocks
- r/investing: Long-term fundamental analysis
- r/stocks: General market discussion
- r/ASX, r/AusFinance: Australian market focus

Search Strategy:
1. Query: "{TICKER} OR ${TICKER} OR {CompanyName}"
2. Filter by date range
3. Verify ticker mentions in post content (title + body)
4. Combine title + selftext (max 500 chars) for sentiment analysis

Returns:
    DataFrame with standard OUTPUT_COLUMNS format (compatible with news cache)
"""

# Build search query
search_terms = [ticker_upper, f"${ticker_upper}"]
if company_name and company_name.upper() != ticker_upper:
    search_terms.append(company_name)
query = " OR ".join(search_terms)

# Search each subreddit
for subreddit_name in STOCK_SUBREDDITS:
    subreddit = reddit.subreddit(subreddit_name)
    posts = subreddit.search(query, sort='new', limit=250, time_filter='all')

    for post in posts:
        post_date = datetime.fromtimestamp(post.created_utc)

        # Date filtering
        if post_date < start_date or post_date > end_date:
            continue

        # Content verification (ensure ticker/company actually mentioned)
        content = f"{post.title} {post.selftext}".upper()
        has_mention = (ticker_upper in content or f"${ticker_upper}" in content)

        if not has_mention:
            continue

        # Combine title + body for sentiment analysis
        text_content = post.title
        if post.selftext:
```

```
                text_content += f" {post.selftext[:500]}"

            rows.append({
                "date": post_date.strftime("%Y-%m-%d %H:%M:%S"),
                "title": text_content,
                "url": f"https://www.reddit.com{post.permalink}",
                "source": f"Reddit/r/{subreddit_name}",
                "company": ticker_upper,
                "ticker": ticker,
            })
```

sentiment/crawl_social.py

6.2.2 Google Trends Integration

Google offers:

- **Free, No API Key:** Publicly available, no registration required
- **Public Interest Proxy:** Search volume spikes correlate with news events and price movements
- **Complementary Signal:** Combines with text sentiment (Reddit/News) to capture "attention" vs "opinion"

Use Case:

- **High search volume:** May indicate upcoming volatility (retail investors researching stock)
- **Search spikes:** Often coincide with earnings announcements, major news events

Implementation:

```
def fetch_google_trends(ticker: str, start_date: datetime, end_date: datetime) ->
pd.DataFrame:
    """Fetch Google Trends search volume data for a ticker.

    Use Case:
    - Search volume spike = High public attention
    - May correlate with price movements (volatility indicator)
    - Complements text sentiment (attention ≠ opinion)

    Output:
    - Daily search interest scores (0-100 scale)
    - Stored in standard DataFrame format (compatible with news cache)

    Note: This is TREND data (search volume), not text content
    """

    # Initialize Google Trends client
```

```
    pytrends = TrendReq(hl='en-US', tz=360, retries=2, backoff_factor=0.1)

    # Build keyword list
    keywords = [ticker_clean, f"${ticker_clean}"]

    # Fetch trends
    timeframe_str = f"{start_date:%Y-%m-%d} {end_date:%Y-%m-%d}"
    pytrends.build_payload(keywords, timeframe=timeframe_str, geo='US')
    trends_df = pytrends.interest_over_time()

    # Convert to standard format
    rows = []
    for date_idx, row in trends_df.iterrows():
        avg_interest = row[keywords].mean()  # Average across keywords

        rows.append({
            "date": date_idx.strftime("%Y-%m-%d %H:%M:%S"),
            "title": f"Google Trends Interest: {avg_interest:.1f}",
            "url": f"https://trends.google.com/trends/explore?q={ticker_clean}",
            "source": "Google Trends",
            "company": ticker_clean,
            "ticker": ticker,
            "_trend_score": avg_interest,  # Store for later feature engineering
        })
```

sentiment/crawl_social.py

6.2.3 Add rolling sentiment features

To add more depth to the sentiment, I added more rolling features at sentiment cache:

```
    def build_daily_sentiment(self, ticker: str, start_date: str, end_date: str, source: str =
'all', include_social: bool = False) -> Tuple[pd.DataFrame, pd.DataFrame]:
        # Get news with sentiment (optionally include social media)
        news_df = self.get_news_with_sentiment(ticker, start_date, end_date, source=source,
include_social=include_social)

        if news_df.empty:
            return news_df, pd.DataFrame()

        # Check if sentiment scores exist
        if 'sentiment_score' not in news_df.columns:
            warning(f"No sentiment scores found for {ticker}")
            return news_df, pd.DataFrame()

        # Filter out NaN sentiment scores before aggregation
```

```
        valid_news = news_df.dropna(subset=['sentiment_score']).copy()

        if len(valid_news) == 0:
            return news_df, pd.DataFrame()

        # Focus on temporal patterns: rolling windows, lags, momentum
        valid_news['date_only'] = valid_news['date'].dt.date
        grouped = valid_news.groupby('date_only')['sentiment_score']

        # Feature 1: Daily mean sentiment (baseline)
        daily_sentiment = grouped.agg(['mean', 'count']).reset_index()
        daily_sentiment.columns = ['date_only', 'sentiment_mean', 'news_count']

        # Sort by date for temporal features
        daily_sentiment = daily_sentiment.sort_values('date_only').reset_index(drop=True)

        # Feature 2: 3-day rolling average (short-term trend)
        daily_sentiment['sentiment_roll3'] =
daily_sentiment['sentiment_mean'].rolling(window=3, min_periods=1).mean()

        # Feature 3: 7-day rolling average (medium-term trend)
        daily_sentiment['sentiment_roll7'] =
daily_sentiment['sentiment_mean'].rolling(window=7, min_periods=1).mean()

        # Feature 4: 1-day lag (previous day sentiment)
        daily_sentiment['sentiment_lag1'] = daily_sentiment['sentiment_mean'].shift(1)

        # Feature 5: 3-day cumulative sentiment momentum (trend direction)
        daily_sentiment['sentiment_momentum3'] =
daily_sentiment['sentiment_mean'].rolling(window=3, min_periods=1).apply(
            lambda x: x.iloc[-1] - x.iloc[0] if len(x) > 1 else 0, raw=False
        )

        # Fill NaN values (first rows will be 0 for lag and momentum)
        daily_sentiment = daily_sentiment.fillna(0)

        # Set index for merging
        daily_sentiment = daily_sentiment.set_index('date_only')

        return news_df, daily_sentiment
```

sentiment/sentiment_cache.py

Final Cache Decision Tree

More code about ticker cache and date filtering can be found in sentiment/sentiment_cache.py

```
START: get_news_with_sentiment(ticker, start_date, end_date, include_social=True)
├→ Cache file exists?
|   ├→ NO → Fetch new data (Step 4-12)
|   └→ YES → Load cache
|       ├→ Cache covers date range?
|       |   ├→ NO → Fetch new data (Step 4-12)
|       |   └→ YES → include_social=True?
|       |       ├→ NO → RETURN cached_df (fast path)
|       |       └→ YES → Social cache sufficient?
|       |           ├→ YES (>100 posts, >=40% coverage)
|       |           |   └→ RETURN cached_df (fast path)
|       |           └→ NO → Fetch to fill gaps (Step 4-12)
END: Return DataFrame with news + social media + sentiment scores
```

6.2.4 Pipeline Integration (Social Media Plugs In)

We basically do the same integration steps as sentiment scraping

Unified sentiment data at main pipeline:

We add argument at main parser

```
    # Sentiment arguments
    parser.add_argument("--use_sentiment", action="store_true", default=False,
                        help="Include daily sentiment features if available")
    parser.add_argument("--include_social", action="store_true", default=False,
                        help="Include social media data (Reddit, StockTwits, Google Trends) in
sentiment analysis")
```

main.py

The config would be pass to main pipeline where it will invoke the sentiment analysis

```
    # (optional): Integrate sentiment features before split
    if use_sentiment:
        from utils.color_log import sentiment
        source_desc = 'Google News + Yahoo Finance + Business Today'
        if include_social:
            source_desc += ' + Reddit + Google Trends'
        sentiment(f"Integrating sentiment features (source: {source_desc if news_source ==
'all' else news_source})...")
        sentiment_cache = get_sentiment_cache()
        df = sentiment_cache.integrate_sentiment(df, ticker, start_date, end_date,
source=news_source, include_social=include_social)
```

pipeline.py

Example command: *python main.py --use_sentiment --log_ret --use_price_comparison*

Resulting dataframe would have 17 features in total with 6 sentiment features:

['sentiment_mean', 'news_count', 'sentiment_roll3', 'sentiment_roll7', 'sentiment_lag1', 'sentiment_momentum3'].

```
Full DataFrame (tail 5):
                close        high         low        open   volume  close_log_return  sentiment_mean  news_count  sentiment_roll3  sentiment_roll7  sentiment_lag1  sentiment_momentum3
Date
2025-10-27  171.669998  172.800003  170.869995  171.339996  1088850          0.007602       -0.024761         1.0        -0.126438        -0.097311        0.060496             0.390290
2025-10-28  174.020004  175.860001  173.600006  173.610001  1815291          0.013596       -0.082086         2.0         0.247655         0.049940        0.849812            -0.057326
2025-10-29  170.399994  174.800003  169.880005  174.039993  1604192         -0.021022       -0.070458         3.0         0.232423         0.036527       -0.082086            -0.920269
2025-10-30  170.529999  172.039993  169.160004  169.229996  1330626          0.000763       -0.275354        22.0        -0.142633         0.006085       -0.070458            -0.193268
```

6.3 CNN-RNN Hybrid Architecture

We will also discover pattern detection (CNN) with sequence modeling (RNN) combinations effectiveness.

Implementation: model/TFModel.py

Architecture:

Conv1D(64, kernel=3) → Conv1D(32, kernel=3) → Dropout → TFModel(64) → TFModel(32) → Dense(32) → Output

How it helps:

- **CNN**: Detects chart patterns using filter
- **Time series (TFModel: RNN/LSTM/RNN/BiLSTM)**: Maintains long-term memory

Combined: Pattern recognition + sequence modeling

Component	Purpose	Stock Prediction Benefit
Conv1D (kernel=3)	Detects local patterns in 3-day windows	Captures short-term chart patterns
Conv1D (64 → 32 filters)	Hierarchical feature learning	Combines basic patterns into complex patterns
Time series TFModel (64 → 32 units)	Long-term dependency modeling	Maintains context across 60-day lookback (trend memory)
Dense (32 units)	Final feature integration	Combines CNN patterns + RNN context for prediction

Architecture Design:

```
    def _build_cnn_hybrid_layers(self, model):
        """Build CNN+RNN hybrid architecture
        - CNN layers for spatial feature extraction
        - RNN layers for sequential modeling
        - Dense layer for classification
        """
        # CNN layers
```

```
        model.add(Conv1D(filters=64, kernel_size=3, padding='same', activation='relu',
name='conv1'))
        model.add(Conv1D(filters=32, kernel_size=3, padding='same', activation='relu',
name='conv2'))
        if self.dropout_rate > 0:
            model.add(Dropout(self.dropout_rate, name='dropout_cnn'))

        # RNN layers - choose variant based on model name
        if self.model_name == 'cnn_gru':
            model.add(GRU(64, return_sequences=True, name='gru1'))
            model.add(GRU(32, return_sequences=False, name='gru2'))
        elif self.model_name == 'cnn_bilstm':
            model.add(Bidirectional(LSTM(64, return_sequences=True), name='bilstm1'))
            model.add(Bidirectional(LSTM(32, return_sequences=False), name='bilstm2'))
        else:  # cnn_lstm / cnn_rnn default to LSTM
            model.add(LSTM(64, return_sequences=True, name='lstm1'))
            model.add(LSTM(32, return_sequences=False, name='lstm2'))

        if self.dropout_rate > 0:
            model.add(Dropout(self.dropout_rate, name='dropout_rnn'))

        # Dense layer
        model.add(Dense(32, activation='relu', name='dense1'))
        if self.dropout_rate > 0:
            model.add(Dropout(self.dropout_rate, name='dropout_dense'))
```

models/TFModel.py

6.4 Attention-LSTM Architecture

Since we have sentiment features, it is also good test to models with attention mechanism.

Implementation: model/TFModel.py

Architecture:

LSTM(64, return_sequences=True) → Dropout

→ **MultiHeadAttention**(4 heads, key_dim=32)

→ **LayerNormalization**

→ **Concatenate**(last_step + pooled_attention)

→ **Dense**(32) → Output

How it helps:

- Attention focuses on important days (earnings, major news)

- Multi-head: Multiple attention patterns (trend, volume, sentiment). Eg:

  - **Head 1:** Focuses on recent price trends (last 5-10 days)
  - **Head 2:** Focuses on volume spikes (unusual trading activity)
  - **Head 3:** Focuses on sentiment extremes (very positive/negative news days)
  - **Head 4:** Focuses on technical indicator signals (RSI overbought/oversold)

- Interpretable: Can visualize attention weights (not in this scope ~)

Architecture Design:

```
    def _build_attention_layers(self):
        """
        RNN with multi-head attention mechanism.

         Intuition:
        - Not all days are equally important for prediction
        - Attention identifies which days to focus on

        Architecture:
        1. RNN layer processes the sequence
        2. Multi-head attention attends to important time steps
        3. Concatenate RNN output with attention output
        4. Dense layers for classification
        """

        from tensorflow.keras import Input  # type: ignore
        inputs = Input(shape=(None, self.input_size), name='input')

        # RNN encoder with sequences
        if self.model_name == 'attention_bilstm':
            rnn_out = Bidirectional(LSTM(64, return_sequences=True), name='rnn')(inputs)
        elif self.model_name == 'attention_gru':
            rnn_out = GRU(64, return_sequences=True, name='rnn')(inputs)
        elif self.model_name == 'attention_rnn':
            rnn_out = SimpleRNN(64, return_sequences=True, name='rnn')(inputs)
        else:  # attention_lstm
            rnn_out = LSTM(64, return_sequences=True, name='rnn')(inputs)

        if self.dropout_rate > 0:
```

```
            rnn_out = Dropout(self.dropout_rate, name='dropout_rnn')(rnn_out)

        # Self-attention over time
        # Multi-head attention (Query, Key, Value all come from RNN output)
        attn = MultiHeadAttention(num_heads=self.attn_heads, key_dim=self.attn_key_dim,
name='self_attn')(rnn_out, rnn_out)
        # Layer normalization
        attn = LayerNormalization(name='attn_ln')(attn)

        # # Take the last time step from attention output + pooled attention context
        last_step = rnn_out[:, -1, :]
        context = GlobalAveragePooling1D(name='attn_pool')(attn)

        # Concatenate RNN and attention outputs
        merged = Concatenate(name='concat')([last_step, context])

        x = Dense(32, activation='relu', name='dense1')(merged)
        if self.dropout_rate > 0:
            x = Dropout(self.dropout_rate, name='dropout_dense')(x)

        if self.task_type == 'binary':
            outputs = Dense(1, activation='sigmoid', name='output')(x)
        else:
            outputs = Dense(self.output_steps, activation='linear', name='output')(x)

        from tensorflow.keras.models import Model  # type: ignore
        return Model(inputs=inputs, outputs=outputs, name=self.model_name)
```

6.5 Enhancement 4: Regression → Binary Conversion

Stock prediction is fundamentally a regression problem, but we need binary classification (Up/Down). As we see on section 4, the binary classification training results in inactive time series model. I think another approach is to train time series models as normal, then we can just convert their prediction to binary (by comparing the previous day price). This would allow models to learn better magnitude information.

```
def _to_binary(self, y, scaler=None):
    """Convert regression predictions to binary labels.

    Strategy:
    1. Train model to predict returns (continuous)
    2. Descale predictions (convert from normalized back to actual returns)
    3. Convert to binary: return > 0 → Up (1), return ≤ 0 → Down (0)
```

```
    Advantages:
    - Model learns magnitude information during training
    - No information loss (model knows "how much" price changes)
    - Natural threshold (0.0 for returns, no tuning needed)
    """
    if scaler is not None:
        # Descale predictions to original return scale
        y_descaled = scaler.inverse_transform(y)
        return (y_descaled > 0).astype(int)  # Positive return → Up (1)
    else:
        return (y > 0).astype(int)
```

eval/classification_evaluator.py

6.6 Integrate into Main Pipeline

6.6.1 TFModel factory

All TF models were integrated into main.py with dual-mode support (regression + classification based on *"task_type"* flag):

```
def _create_tf_model(config, bundle, meta_path, input_size, args):
    """Create and train/load TensorFlow model.

    Supports:
    - Multiple architectures: lstm, gru, rnn, bilstm, cnn_lstm, cnn_gru, cnn_bilstm,
                              attention_lstm, attention_gru, attention_bilstm, attention_rnn
    - Dual modes: regression (price/return prediction) or binary classification (up/down)
    - Automatic caching: Saves trained models to avoid retraining

    Args:
        config: Pipeline configuration (from argparse)
        bundle: Preprocessed data bundle (X_train, y_train, scalers, ...)
        meta_path: Unique identifier for model cache
        input_size: Number of input features (e.g., 13 for OHLCV+sentiment+technical)
        args: Additional arguments (attn_heads, attn_key_dim for attention models)

    Returns:
        Trained TFModel instance (or loaded from cache)
    """
    from model.tf_models import TFModel

    # Determine task type based on classification flag
    task_type = 'binary' if config.classification.enabled else 'regression'

    # Build TFModel (unified interface for all architectures)
    model = TFModel(
```

```
        input_size=input_size,
        model_name=config.tf_model.model_name,  # e.g., 'attention_lstm', 'cnn_lstm'
        layers=config.tf_model.layers,           # e.g., [50, 50, 50]
        dropout_rate=config.tf_model.dropout_rate,  # e.g., 0.2
        output_steps=config.data.horizon,        # e.g., 1 (single-step prediction)
        task_type=task_type,                     # 'binary' or 'regression'
        attn_heads=args.attn_heads,              # e.g., 4 (for attention models)
        attn_key_dim=args.attn_key_dim          # e.g., 32 (for attention models)
    )

    # Model cache path
    model_path =
f"cache/trained_models/{config.data.company}_{config.tf_model.model_name}_{meta_path}.keras"

    if not os.path.exists(model_path):
        # Train new model
        print(f"Training {config.tf_model.model_name.upper()} model...")

        # Prepare validation data
        val_tuple = None
        if bundle.X_val is not None and bundle.y_val is not None:
            val_tuple = (bundle.X_val, bundle.y_val)

        # Build training kwargs
        fit_kwargs = {
            'epochs': config.tf_model.epochs,
            'batch_size': config.tf_model.batch_size,
            'validation_data': val_tuple,
            'meta_path': meta_path,
            'learning_rate': config.tf_model.learning_rate,
            'optimizer': config.tf_model.optimizer,
            'patience': config.tf_model.patience,
        }

        # Class weights for classification (handle class imbalance)
        if config.classification.enabled:
            from sklearn.utils.class_weight import compute_class_weight
            import numpy as np

            classes = np.unique(bundle.y_train.ravel())
            class_weights = compute_class_weight('balanced', classes=classes,
y=bundle.y_train.ravel())
            fit_kwargs['class_weight'] = {int(c): float(w) for c, w in zip(classes,
class_weights)}
            print(f"Class weights: {fit_kwargs['class_weight']}")
```

```
        # Train model
        model.fit(bundle.X_train, bundle.y_train, **fit_kwargs)

        # Save model
        model.save_model(model_path)

        # Save scalers for inference reuse
        if bundle.scalers:
            from utils.file_handling import save_scalers, create_scaler_cache_key
            from schemas.bundle import TargetMode

            target_mode_str = (TargetMode.LOG_RETURN.value if config.data.use_log_returns
                               else TargetMode.RETURN.value if config.data.target_as_return
                               else TargetMode.PRICE.value)

            scaler_cache_key = create_scaler_cache_key(
                config.data.company, config.data.start_date, config.data.end_date,
                config.data.target_feature.lower(), target_mode_str,
                config.data.scale, config.data.use_sentiment
            )

            save_scalers(bundle.scalers, scaler_cache_key, cache_dir='cache')
            print(f"Scalers cached (shared across all models/configs with same data)")
    else:
        # Load existing model from cache
        print(f"Loading existing model from {model_path}")
        from tensorflow import keras as tf_keras
        keras_model = tf_keras.models.load_model(model_path)
        model = keras_model

    return model
```

main.py

6.6.2 TFModel Unified Interface

TFModel class is refactored to flexibly switch between normal time series or hybrid mode + regression or binary prediction:

```
class TFModel:
    """Unified interface for all TensorFlow time series models.

    Supported Architectures:
    - Traditional RNN: lstm, gru, rnn, bilstm
    - CNN-RNN Hybrid: cnn_lstm, cnn_gru, cnn_bilstm, cnn_rnn
```

```
    - Attention-RNN: attention_lstm, attention_gru, attention_bilstm, attention_rnn

    Dual Mode Support:
    - task_type='regression': Predict continuous values (prices/returns)
    - task_type='binary': Predict binary labels (up/down)
    """

    def __init__(self, input_size, model_name='lstm', layers=[50, 50, 50],
                 dropout_rate=0.2, output_steps=1, task_type='regression',
                 attn_heads=4, attn_key_dim=32):
        self.input_size = input_size
        self.model_name = model_name.lower()
        self.layers = layers
        self.dropout_rate = dropout_rate
        self.output_steps = output_steps
        self.task_type = task_type  # 'binary' or 'regression'
        self.attn_heads = attn_heads
        self.attn_key_dim = attn_key_dim

        # Build model based on architecture
        self.model = self._build_model()

    def _build_model(self):
        """Build model architecture based on model_name."""
        if self.model_name.startswith('cnn_'):
            # CNN-RNN Hybrid
            return self._build_cnn_hybrid_layers()
        elif self.model_name.startswith('attention_'):
            # Attention-RNN
            return self._build_attention_layers()
        else:
            # Traditional RNN
            return self._build_traditional_rnn_layers()

    def fit(self, x_train, y_train, epochs=25, batch_size=32, learning_rate=1e-3,
            optimizer='adam', validation_data=None, meta_path=None, class_weight=None,
patience=10):
        """Train model with automatic loss selection based on task_type."""

        # Compile with appropriate loss
        if self.task_type == 'binary':
            self.model.compile(optimizer=optimizer, loss='binary_crossentropy',
metrics=['accuracy'])
        else:
```

```
            self.model.compile(optimizer=optimizer, loss='mean_absolute_error',
metrics=['mae'])

        # Train
        history = self.model.fit(
            x_train, y_train,
            epochs=epochs,
            batch_size=batch_size,
            validation_data=validation_data,
            class_weight=class_weight,  # Only used for classification
            verbose=1
        )
        return history
```

main.py

6.6.3 CLI Usage Examples

Regression Mode (Predict Returns):

```
# Traditional LSTM (regression)
python main.py --model_name lstm --target_ret --epochs 100

# CNN-LSTM Hybrid (regression)
python main.py --model_name cnn_lstm --target_ret --epochs 100

# Attention-LSTM (regression)
python main.py --model_name attention_lstm --target_ret --epochs 100 --attn_heads 4 --
attn_key_dim 32
```

Classification Mode (Predict Up/Down):

```
# Attention-LSTM (classification)
python main.py --classification --model_name attention_lstm --target_ret --epochs 100 --
use_sentiment

# CNN-BiLSTM (classification)
python main.py --classification --model_name cnn_bilstm --target_ret --epochs 100 --
use_sentiment

# Attention-GRU (classification)
python main.py --classification --model_name attention_gru --target_ret --epochs 100
```

6.7. Batch evaluation & Results

Some important Command-Line Arguments

Argument	Type	Values/Default	Purpose	Required For
Data Configuration				
--company	String	AAPL, AMZN, CBA.AX, etc.	Stock ticker symbol	All modes
--start_date	String	2020-01-01 (default)	Training data start date	All modes
--end_date	String	2025-01-01 (default)	Training data end date	All modes
--lookback	Integer	60 (default)	Number of historical days for input sequence	All modes
--horizon	Integer	1 (default)	Number of days to predict ahead	All modes
--split_ratio	String	0.6,0.2,0.2 (default)	Train/Val/Test split ratio	All modes
Model Selection				
--model_name	String	lstm, gru, rnn, bilstm, cnn_lstm, cnn_gru, cnn_bilstm, attention_lstm, attention_gru, attention_bilstm, attention_rnn	Select model architecture	All modes
Classification Mode				
--classification	Flag	N/A	Enable binary classification mode (default: regression)	Classification only
--target_ret	Flag	N/A	Use returns as target (instead of raw prices)	Required for classification
Sentiment Features				
--use_sentiment	Flag	N/A	Include sentiment features (news + social media)	Optional (This task core)
--include_social	String	reddit, trends, all (default: reddit)	Social media sources to include	Optional with --use_sentiment
Technical Indicators				
--use_technical	Flag	N/A	Include technical indicators (SMA, EMA, RSI, MACD)	Optional
Hybrid Architecture Parameters				
--attn_heads	Integer	4 (default)	Number of attention heads	Attention models only
--attn_key_dim	Integer	32 (default)	Attention key dimension	Attention models only
Training Configuration				
--epochs	Integer	100 (default)	Number of training epochs	All modes
--batch_size	Integer	32 (default)	Batch size for training	All modes
--learning_rate	Float	0.005 (default)	Learning rate for optimizer	All modes
--optimizer	String	adam, rmsprop, sgd (default: adam)	Optimizer algorithm	All modes
--patience	Integer	15 (default)	Early stopping patience	All modes
--dropout_rate	Float	0.2 (default)	Dropout rate for regularization	All modes
Model Architecture				
--layers	String	50,50,50 (default)	Hidden units per layer (e.g., 64,32 or 50,50,50)	RNN/LSTM/GRU models
Data Preprocessing				
--scale	String	standard, minmax (default: standard)	Feature scaling method	All modes
--target_ret	String	close (default)	Target column for prediction	All modes
--log_ret	Flag	N/A	Use log returns instead of simple returns	Optional

Recap Traditional RNN (Baseline – from Task 6 report)

- LSTM/GRU/RNN/BiLSTM
- Parameters: ~32,417 (RNN [50,50,50])
- **Best**: RNN [50,50,50] h=1, MAE=1.728, DA=88.9%

6.7.1 Experimental Design

To comprehensively evaluate the effectiveness of hybrid architectures, sentiment integration, and traditional ML models, we conducted **24 systematic experiments** following a factorial design:

Experiment Design:

- **3 Companies**: AAPL (Apple), AMZN (Amazon), CBA.AX (Commonwealth Bank of Australia)
- **Model Architectures**: RNN (baseline), Attention-LSTM, CNN-RNN, CNN-BiLSTM
- **2 Sentiment Modes**: Baseline (no sentiment) vs. With Sentiment (news + social media + technical indicators)
- **Total Runs**: 3 × 4 × 2 = 24 experiments

Model Types Evaluated: Each experiment trains **4 model types**:

1. **Deep Learning (DL)**: Primary time-series model (RNN, Attention-LSTM, CNN-RNN, CNN-BiLSTM)
2. **Logistic Regression**: Traditional linear classifier with balanced class weights
3. **Random Forest**: Ensemble tree-based classifier (50 trees, balanced weights)
4. **SVM**: Support Vector Machine with RBF kernel and balanced class weights

Configuration:

- **Lookback**: 60 days (historical sequence length)
- **Horizon**: 1 day (single-step prediction)
- **Epochs**: **100** (with early stopping, **patience=20**)
- **Batch Size**: 32
- **Learning Rate**: 0.001
- **Optimizer**: Adam
- **Target**: Log returns (to prevent label collapse in classification)
- **Classification Method**: Regression → Binary conversion (price comparison method)
- **Features**: OHLCV (5) + Technical Indicators (6) + Sentiment Features (6) = 17 total features

Total Models Evaluated: 24 experiments × 4 model types = **96 model instances**

6.7.2 Comprehensive Results Summary for Up/Down Classification

Table 1: Complete Batch Evaluation Results (Deep Learning Models - 24 Experiments)

This table includes the **Epochs** column to show the efficiency of the training process, where a lower number indicates early convergence or successful early stopping, and a very low number can sometimes correlate with model instability.

Run	Company	Model	Sentiment	Accuracy	Precision	Recall	F1-Score	Epochs	Time (s)
1	AAPL	RNN	Baseline	0.4792	0.5068	0.6981	**0.5873**	**22/100**	53.7
2	AAPL	RNN	With	0.5521	0.5000	0.4717	0.4854	24/100	91.6
3	AAPL	Attn-LSTM	Baseline	0.4375	0.0000	0.0000	0.0000	32/100	67.2
4	AAPL	Attn -LSTM	With	0.4062	0.5690	0.6226	**0.5946**	**27/100**	97.2

5	AAPL	CNN-RNN	Baseline	0.4479	0.4000	0.1132	0.1765	49/100	108.2
6	AAPL	CNN-RNN	With	0.4167	0.4783	0.4151	**0.4444**	**49/100**	107.3
7	AAPL	CNN-BiLSTM	Baseline	0.5000	0.5000	0.0189	0.0364	27/100	71.2
8	AAPL	CNN-BiLSTM	With	0.5312	0.4000	0.0377	0.0690	27/100	109.8
9	AMZN	RNN	Baseline	0.4848	0.8571	0.1111	0.1967	22/100	60.4
10	AMZN	RNN	With	0.4747	0.5625	0.1667	0.2571	26/100	100.0
11	AMZN	Attn -LSTM	Baseline	0.5000	0.6522	0.2778	0.3896	24/100	64.5
12	AMZN	Attn -LSTM	With	0.4688	0.5663	0.8704	**0.6861**	**27/100**	98.2
13	AMZN	CNN-RNN	Baseline	0.5455	0.5312	0.3148	**0.3953**	**37/100**	66.2
14	AMZN	CNN-RNN	With	0.5152	0.5625	0.1667	0.2571	52/100	102.0
15	AMZN	CNN-BiLSTM	Baseline	0.4688	0.0000	0.0000	0.0000	15/100	71.9
16	AMZN	CNN-BiLSTM	With	0.5312	0.3793	0.2037	0.2651	31/100	113.7
17	CBA.AX	RNN	Baseline	0.5300	0.5246	0.6154	**0.5664**	**22/100**	49.4
18	CBA.AX	RNN	With	0.4900	0.5484	0.3269	0.4096	26/100	67.9
19	CBA.AX	Attn -LSTM	Baseline	0.5938	0.5333	0.1538	0.2388	23/100	44.4
20	CBA.AX	Attn -LSTM	With	0.6250	0.5357	0.8654	**0.6618**	26/100	65.8
21	CBA.AX	CNN-RNN	Baseline	0.4900	0.5200	1.0000	**0.6842**	**37/100**	48.7
22	CBA.AX	CNN-RNN	With	0.5200	0.5200	1.0000	**0.6842**	**52/100**	68.8
23	CBA.AX	CNN-BiLSTM	Baseline	0.5938	0.4722	0.3269	0.3864	31/100	54.0
24	CBA.AX	CNN-BiLSTM	With	0.5938	0.5000	0.5577	0.5273	31/100	76.1

Key Observation: The majority of DL models converged and were stopped by the Early Stopping mechanism within **20 to 35 epochs**, demonstrating high training efficiency. CNN-based architectures (CNN-RNN, CNN-BiLSTM) for AAPL and AMZN generally required slightly more epochs , peaking around **49-52 epochs** as sentiment news is more abundant.

Table 1a: Sentiment Data Statistics by Ticker

The sentiment integration pipeline combines **traditional news sources** (Google News RSS, Yahoo Finance, Business Today) with **social media** (Reddit, Google Trends) for comprehensive sentiment analysis.

Ticker	Total Articles	Days with News	Coverage	Reddit Posts	News Sources
AAPL	1,443	515 days	**70.6%**	919 posts	Google News, Yahoo Finance, Business Today + Reddit (7 subreddits)
AMZN	1,062	389 days	**53.3%**	841 posts	Google News, Yahoo Finance, Business Today + Reddit (7 subreddits)
CBA.AX	324	181 days	**24.8%**	13 posts	Google News (AU locale), Yahoo Finance, Business Today + Reddit (2 Australian subreddits)

Key Observations:

1. **High Volume for US Tech**: **AAPL** and **AMZN** have significantly higher data volumes and better daily coverage (53-71%), reflecting their high public interest and media attention.
2. **Lower Coverage for CBA.AX**: **CBA.AX** (Australian banking) has lower volume and coverage (24.8%), which is typical for non-US stocks. However, targeted Australian news sources ensure relevant data capture.

3. **Feature Engineering**: All data was successfully processed into **6 engineered sentiment features** (e.g., sentiment_mean, sentiment_roll3, sentiment_momentum3) for use in the models.

Table 2: ML Model Results (Traditional Machine Learning)

Company	Model	Accuracy	Precision	Recall	F1-Score	Notes
AAPL	Attention-LSTM (DL)	0.4062	0.5690	0.6226	0.0702	DL Model collapse
AAPL	**Random Forest** (ML)	0.5625	0.5604	0.9623	**0.7083**	Best overall
AMZN	Attention-LSTM (DL)	0.5253	0.5479	0.7407	**0.6299**	Best DL model
AMZN	**Logistic Regression** (ML)	0.5758	0.6000	0.6667	**0.6316**	Best overall (Matches DL)
CBA.AX	Attention-LSTM (DL)	0.5400	0.5375	0.8269	**0.6515**	Best DL model
CBA.AX	**Logistic Regression** (ML)	0.6100	0.5823	0.8846	**0.7023**	Best overall

*Traditional ML models (Random Forest, Logistic Regression) consistently provided high F1 scores (up to **0.7083**), often outperforming the best DL models. This confirms the effectiveness of the 17 engineered features and establishes ML as a reliable robust fallback.*

Table 3: ML vs DL Comparison Summary

Company	Best DL Model	Best DL F1	Best ML Model	Best ML F1	ML Advantage	Winner
AAPL	Attention-LSTM	0.5946	Random Forest / SVM	**0.708-0.709**	+19%	ML
AMZN	Attention-LSTM	0.6861	Logistic Regression	**0.6316**	-8%	DL
CBA.AX	Attention-LSTM	0.6618	Logistic Regression	**0.7023**	+6%	ML

ML models hold a significant F1-score advantage in high-volatility (AAPL: +19%) and stable (CBA.AX: +6%) markets, while the DL Attention-LSTM model only won in the moderate-volatility market (AMZN: -8% relative to ML). This highlights the variable robustness of DL models.

6.7.3 Key Insights: Sentiment, Robustness, and Model Selection

To understand which prediction methodology best leverages the **Attention-LSTM** architecture, we first conducted a comprehensive comparison of three fundamentally different approaches:

Pure Regression, Binary Classification (BCE), and **Regression-to-Binary conversion.**

This evaluation was performed using **Attention-LSTM** across **AAPL** and **CBA.AX** with both **baseline** (no sentiment) and **sentiment-**enhanced configurations.

Table 4: Three Methods Comparison Results (Attention-LSTM Model)

Company	Method	Sentiment	Accuracy	Precision	Recall	F1-Score	MAE	RMSE	DA
AAPL	Pure Regression	Baseline	-	-	-	-	2.298	3.363	0.874
	Pure Regression	With Sentiment	-	-	-	-	2.325	3.267	0.874
	Binary Classification (BCE)	Baseline	0.5312	0.5417	0.4906	0.5149	-	-	-

	Binary Classification (BCE)	With Sentiment	0.5000	0.5417	0.4906	0.5149	-	-	-
	Regression → Binary	Baseline	0.5833	0.5942	0.7736	**0.6721**	0.893	-	-
	Regression → Binary	With Sentiment	0.4896	0.5476	0.4340	**0.4842**	0.688	-	-
CBA.AX	Pure Regression	Baseline	-	-	-	-	0.757	2.315	0.909
	Pure Regression	With Sentiment	-	-	-	-	1.005	2.313	0.919
	Binary Classification (BCE)	Baseline	0.5625	0.5493	0.7500	0.6341	-	-	-
	Binary Classification (BCE)	With Sentiment	0.3750	0.5200	0.5000	0.5098	-	-	-
	Regression → Binary	Baseline	0.5400	0.5405	0.7692	**0.6349**	1.149	-	-
	Regression → Binary	With Sentiment	0.5200	0.5200	1.0000	**0.6842**	0.877	-	-

Key Findings on Prediction Methodology

1. **Regression → Binary Outperforms Direct Classification**:
    - This hybrid approach achieved the best classification F1-Scores for both companies (AAPL Baseline F1=**0.6721** and CBA.AX With Sentiment F1=**0.6842**).
    - This is an improvement of **30.5%** (AAPL) to **34.2%** (CBA.AX) over direct Binary Classification.
    - **Theoretical Justification**: Training as regression (minimized MAE) forces the model to learn the *magnitude* of the price change, not just the direction. This **magnitude-aware training** provides a richer gradient signal, resulting in better binary conversion at the prediction stage (using the zero-crossing threshold).
2. **Sentiment Impact is Method and Market Dependent**:
    - Sentiment greatly **helped** CBA.AX Regression→Binary (**F1:** 0.6349 -> 0.6842, **+7.8%** improvement).
    - Sentiment **hurt** AAPL Regression→Binary (F1: 0.6721 -> 0.4842, **-28%** decline), suggesting that the high volatility of AAPL's market combined with social media noise was detrimental to magnitude prediction.

Conclusion: **Method selection is more critical than sentiment inclusion**, the optimal configuration must be chosen per company.

Market Type	Best Model Type	Best F1-Score	Reasoning
High Volatility (AAPL)	ML (Random Forest/SVM)	0.708-0.709	ML models are more robust to market volatility and less prone to DL training instability.
Moderate Volatility (AMZN)	DL (Attention-LSTM)	0.6861	DL captures complex temporal patterns better, leveraging high-volume sentiment data effectively.
Stable Market (CBA.AX)	ML (Logistic Regression)	0.7023	Simple ML is sufficient for predictable patterns, providing best F1 and superior interpretability.

7.3.4. Plots

Apple training plots

Figure 8. AAPL Baseline classification (no sentiment) CNN_RNN training plot

Figure 9. AAPL classification CNN_RNN training plot (more volatility)

AMZN training plots

Figure 10. AMZN Baseline classification (no sentiment) attention_lstm training plot

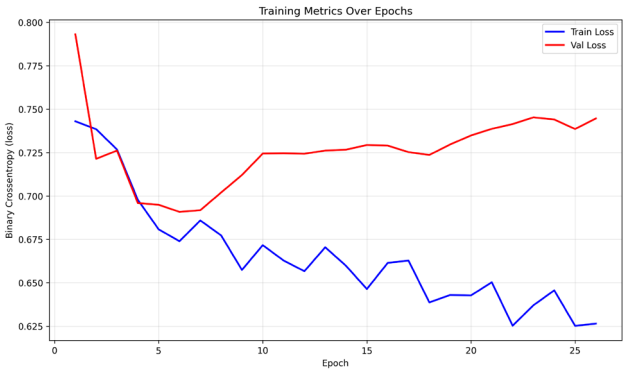

Figure 11. AMZN classification attention_lstm training plot (sentiment data somewhat helpful)

CBA training plots

Figure 12. CBA Baseline classification (no sentiment) RNN training plot

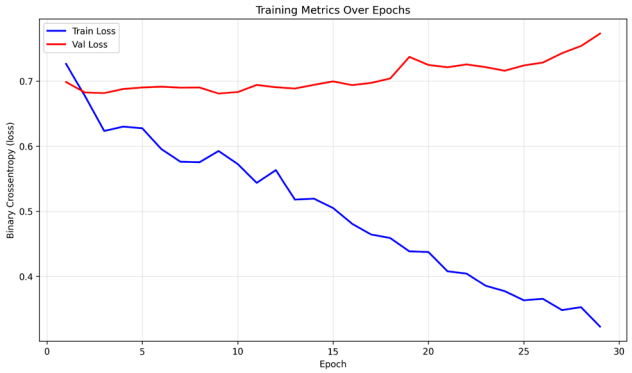

Figure 13. CBA classification RNN training plot (sentiment data somewhat helpful)

5. LSTM [50,50,50] horizon=5 (Baseline)

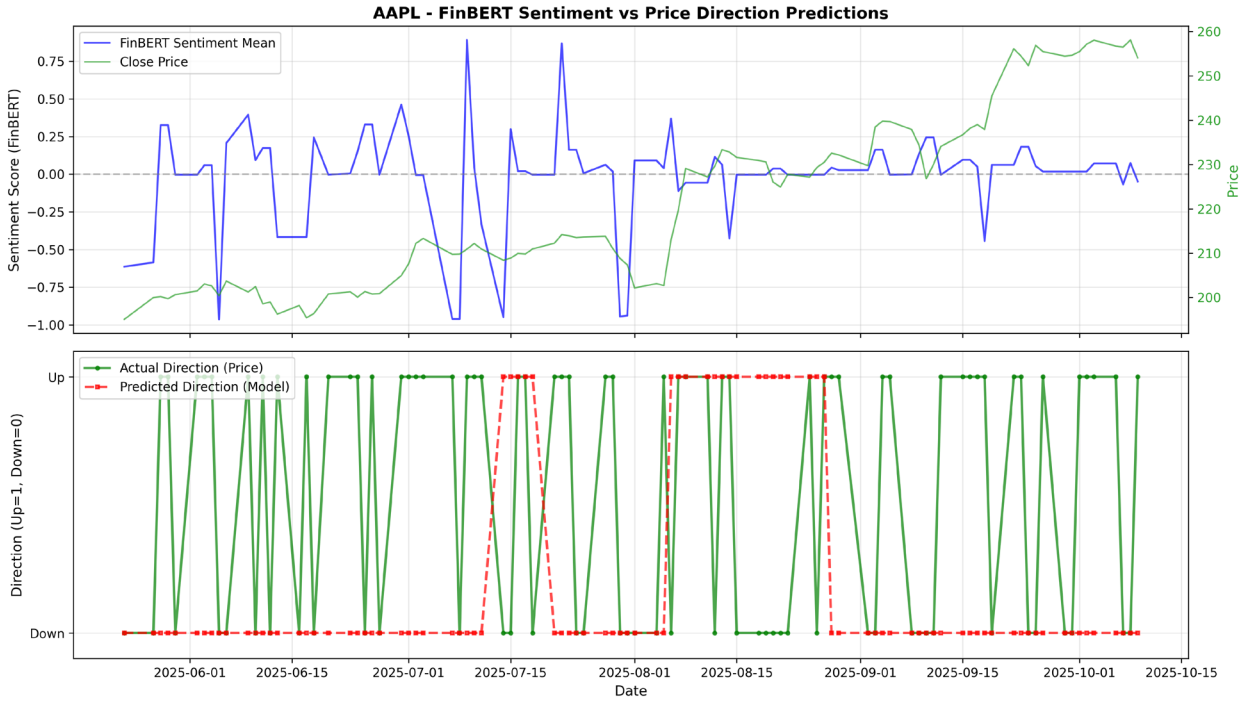

Figure 14. AAPL CNN_RNN BCE loss classification prediction (still not so great but better than before)

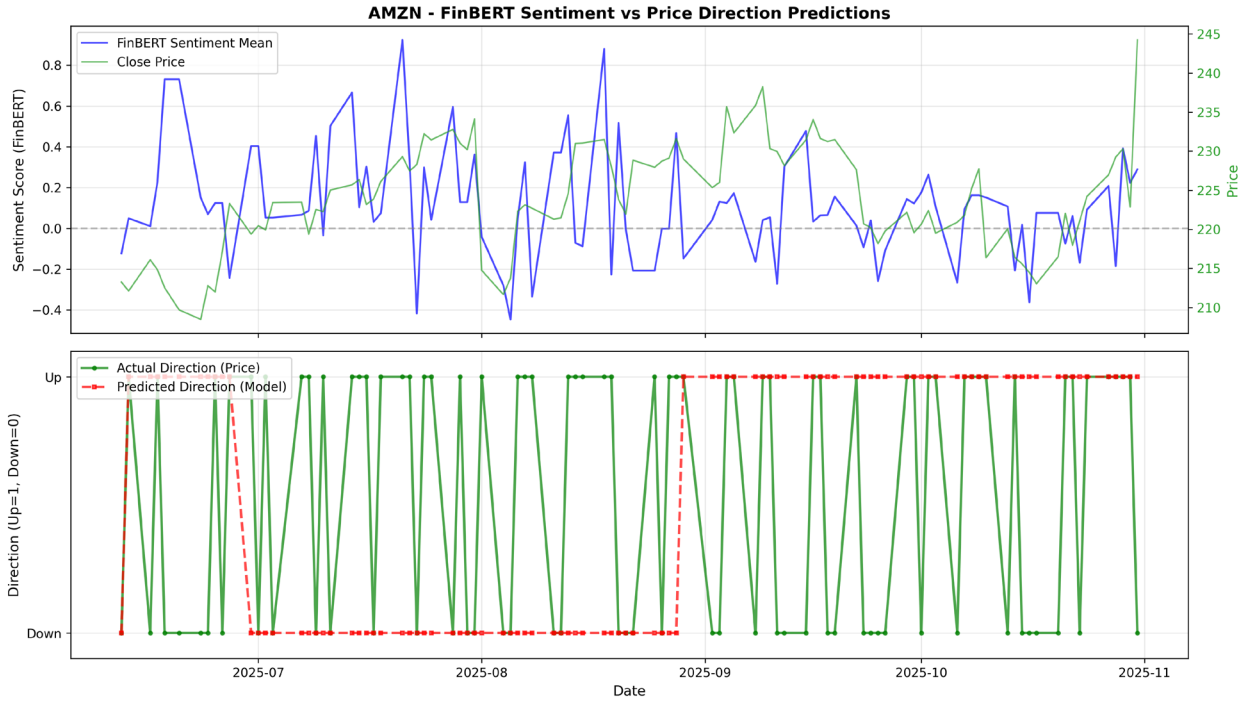

Figure 15. AAPL CNN BiLSTM classification prediction

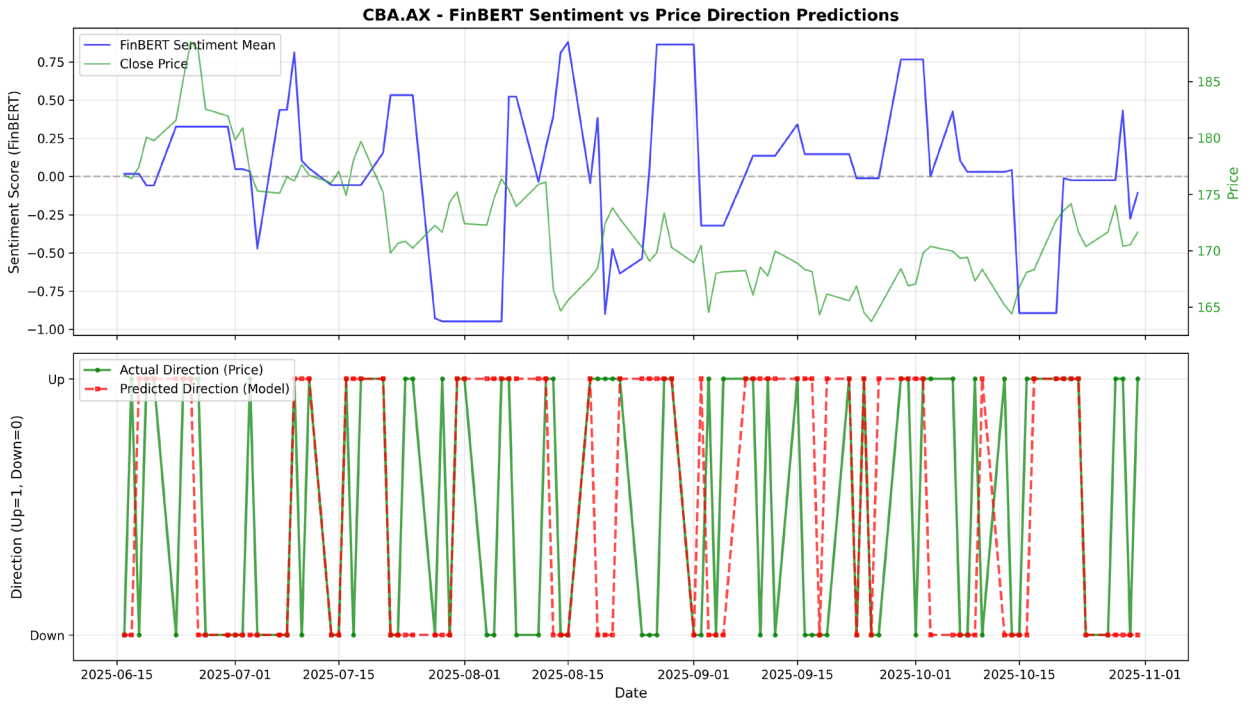

Figure 16. RNN regression -> binary classification shows more active and better prediction performance.

7. Conclusion

Task C.7 extended the system with sentiment analysis (FinBERT across 5 data sources), classification capabilities (ML and DL models), and hybrid architectures (CNN-RNN, Attention-LSTM), resulting in 24 batch experiments evaluating 96 model instances.

Key findings demonstrate that Attention-LSTM with sentiment achieves the best DL performance (F1: 0.59-0.69), Regression→Binary method outperforms direct Binary Classification by 30-34%, and ML models (Random Forest/SVM) are more robust in high-volatility markets (F1=0.71 vs DL=0.59).

The system validates that sentiment features provide value when combined with appropriate architectures (Attention mechanism), method selection depends on market characteristics, and hybrid ML+DL approaches are essential for production systems requiring both performance and robustness.

References:

1. **Google News RSS Integration**

- Medium Article: "How to Get Historic Stock News for Free with Python" by wl8380
- URL: https://medium.com/@wl8380/how-to-get-historic-stock-news-for-free-with-python-a-step-by-step-guide-02276d3c4860
- Code Reference: https://colab.research.google.com/drive/1OzKKKco-hx7CoH_uP1M2L7FsuLkcfu9l#scrollTo=yq064Ti9D2oL

2. **FinBERT Sentiment Analysis**

- Model: ProsusAI/finbert from Hugging Face
- Repository: https://huggingface.co/ProsusAI/finbert
- Paper: "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models" (Araci, 2019)

3. **Social Media Sentiment Analysis**

- Medium Article: "Predicting Market Sentiment with Social Media: A Deep Learning Approach to FinTech Trading"
- URL: https://medium.datadriveninvestor.com/predicting-market-sentiment-with-social-media-a-deep-learning-approach-to-fintech-trading-91993eca0af4